

Q1. Show each employee's name, salary, and the total salary of ALL employees in the company on every row.

```
SELECT name, salary,  
SUM(salary) OVER() AS total_company_salary  
FROM employees;
```

Q2. Show each employee alongside the maximum salary earned by anyone in the company.

```
SELECT name, salary,  
MAX(salary) OVER() AS company_max  
FROM employees;
```

Q3. Show each employee alongside the minimum salary in the company.

```
SELECT name, salary,  
MIN(salary) OVER() AS company_min  
FROM employees;
```

Q4. Display every employee row with the TOTAL number of employees.

```
SELECT name, dept,  
COUNT(*) OVER() AS total_employees  
FROM employees;
```

Q5. Show each employee's salary and what percentage their salary is of the entire company payroll.

```
SELECT name, salary,  
ROUND(salary * 100.0 / SUM(salary) OVER(), 2) AS pct_of_total  
FROM employees  
ORDER BY pct_of_total DESC;
```

Q6. Order employees from lowest to highest salary and compute a cumulative running total.

```
SELECT name, salary,  
SUM(salary) OVER(ORDER BY salary) AS running_total  
FROM employees;
```

Q7. Show the highest salary seen among all employees hired up to and including each hire year.

```
SELECT name, hire_year, salary,  
MAX(salary) OVER(ORDER BY hire_year) AS max_so_far  
FROM employees  
ORDER BY hire_year;
```

Q8. Track the minimum salary across hire years.

```
SELECT name, hire_year, salary,  
MIN(salary) OVER(ORDER BY hire_year) AS min_so_far  
FROM employees  
ORDER BY hire_year;
```

Q9. For each employee, show how many employees had been hired up to and including their hire year.

```
SELECT name, hire_year,  
COUNT(*) OVER(ORDER BY hire_year) AS employees_hired_so_far  
FROM employees  
ORDER BY hire_year;
```

Q10. Show the cumulative average salary from the lowest-paid employee to the highest.

```
SELECT name, salary,  
ROUND(AVG(salary) OVER(ORDER BY salary), 0) AS running_avg  
FROM employees;
```

Q11. Show each employee with their salary and their department's total salary payout.

```
SELECT name, dept, salary,  
SUM(salary) OVER(PARTITION BY dept) AS dept_total  
FROM employees;
```

Q12. Show each employee alongside the highest salary in their department.

```
SELECT name, dept, salary,  
MAX(salary) OVER(PARTITION BY dept) AS dept_max  
FROM employees;
```

Q13. Show each employee alongside the lowest salary in their department.

```
SELECT name, dept, salary,  
MIN(salary) OVER(PARTITION BY dept) AS dept_min  
FROM employees;
```

Q14. Show each employee's salary, department average, and difference from department average.

```
SELECT name, dept, salary,  
ROUND(AVG(salary) OVER(PARTITION BY dept), 0) AS dept_avg,  
salary - ROUND(AVG(salary) OVER(PARTITION BY dept), 0) AS diff
```

FROM employees;

Q15. Show every employee with the number of people working in their department.

```
SELECT name, dept,  
COUNT(*) OVER(PARTITION BY dept) AS dept_size  
FROM employees  
ORDER BY dept;
```

Q16. Show what percentage each employee's salary contributes to their department's total payroll.

```
SELECT name, dept, salary,  
ROUND(salary * 100.0 / SUM(salary) OVER(PARTITION BY dept), 2) AS pct_of_dept  
FROM employees;
```

Q17. Within each department, show the cumulative salary total ordered by hire year.

```
SELECT name, dept, hire_year, salary,  
SUM(salary) OVER(  
    PARTITION BY dept  
    ORDER BY hire_year  
) AS running_dept_total  
FROM employees  
ORDER BY dept, hire_year;
```

Q18. Within each department, show the highest salary seen so far by hire year.

```
SELECT name, dept, hire_year, salary,  
MAX(salary) OVER(  
    PARTITION BY dept  
    ORDER BY hire_year  
) AS running_dept_max  
FROM employees  
ORDER BY dept, hire_year;
```

Q19. Track the minimum salary within each department over hire years.

```
SELECT name, dept, hire_year, salary,  
MIN(salary) OVER(  
    PARTITION BY dept
```

```
ORDER BY hire_year
) AS running_dept_min
FROM employees
ORDER BY dept, hire_year;
```

Q20. Show the cumulative average salary within each department as new employees are added.

```
SELECT name, dept, hire_year, salary,
ROUND(AVG(salary) OVER(
PARTITION BY dept
ORDER BY hire_year
), 0) AS running_dept_avg
FROM employees
ORDER BY dept, hire_year;
```

Q21. For each department, show the growing headcount over hiring years.

```
SELECT name, dept, hire_year,
COUNT(*) OVER(
PARTITION BY dept
ORDER BY hire_year
) AS cumulative_headcount
FROM employees
ORDER BY dept, hire_year;
```

Q22. Label each employee as Above Avg or Below Avg based on department average.

```
SELECT name, dept, salary,
ROUND(AVG(salary) OVER(PARTITION BY dept), 0) AS dept_avg,
CASE
WHEN salary > AVG(salary) OVER(PARTITION BY dept)
THEN 'Above Avg'
ELSE 'Below Avg'
END AS status
FROM employees;
```